University of Texas at Dallas

Department of Electrical Engineering

EEDG/CE 6306 - Application Specific Integrated Circuit Design

Homework #4

Submission:

(a) HW Report

- **System-level Microarchitecture (30pts)**
- **Top-level specifications (10pts)**
- **Block-level specifications (40pts)**
- **Generative AI for HW design summary and strategy (10pts)**

Submit onto eLearning a zip file (team): <First name #1>_<Last name #1>_<FIrst name #2>_<Last name#2>_HW04.zip

* If you have any question about the homework, you may contact TA

Purpose: Develop system architecture and block-level specifications.

In this assignment we will start to design the block-level microarchitecture inside of the MSDAP. The system-level architecture is shown in Figure 1.

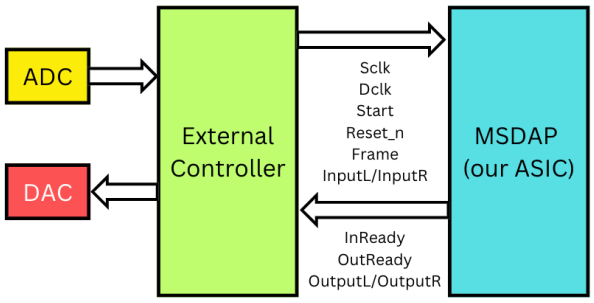


Figure 1. System diagram for MSDAP

(1) Design and draw a system-level microarchitecture diagram for MSDAP. Your microarchitecture may have multiple layers; for example, on the top layer you may have the main controller which manipulates the control signals for

both channels. At a lower layer, for example, you may have the ALU block which is controlled by the main controller. At the lowest level, you may have a one-bit shifter and an accumulator which are controlled by the ALU. An example ALU microarchitecture is included in the Appendix. Your report should include all the block diagrams. You may draw the microarchitecture either with a top down manner or a bottom up manner.

Your microarchitecture should at least include the following modules:

- Memories and Registers
- ALU
- Input interface - Parallel to Serial (P2S) converter
- Output interface - Serial to Parallel (S2P) converter
- Main controller (FSM)

Inside those modules, you are free to define sub-modules in your design. See Appendix for more details.

(1.1) Provide a written explanation of reasons why you chose your microarchitecture.

(2) Complete ASIC specification for MSDAP. The specification should include the specification you wrote for the top-level MSDAP from HW#2. You can refer to this website on how to write a good microarchitecture specification. In HW#3, you wrote the top-level specification.

(2.1) To the top-level spec include the frequency of "SCLK" and how you determine it.

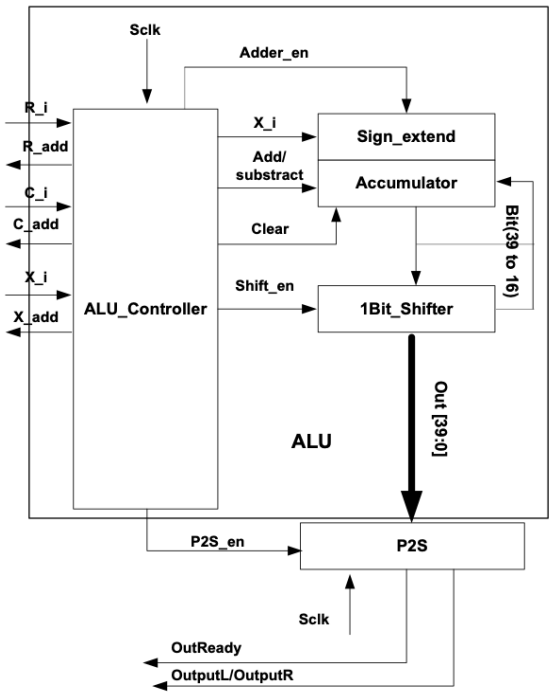
(2.2) Next add block-level specifications. The block-level specifications should include

- Block functional description
- Input/Output signals and descriptions
- Interface requirements
- Performance requirements

(3) Read the article "Leveraging Generative AI for Rapid Design and Verification of a Vector Processor SoC" by Salcedo, McBeth and Achour. Summarize your key learnings about using AI for HW design in <500 words and discuss if you plan to use AI in your MSDAP design. (Btw, if you do use AI you can have a chance to gain bonus points in future assignments 😉)

Appendix

ALU microarchitecture



Example microarchitecture for the ALU module

Interactions of MSDAP and External Controller (testbench)

Sclk (Input)

System clock running at a **frequency of 26.88MHz (you may modify this frequency)** provides the timing reference for the internal and control signals, as well as the output samples. Outputs InReady and OutReady are updated on the rising edge of Sclk.

Dclk (Input)

Data clock running at a **frequency of 768kHz (this frequency is fixed)** provides the timing reference for the input samples.

Start (Input)

When Start is set high, the chip begins to initialize. Start may be asynchronous with Sclk or Dclk.

Reset_n (Input)

When Reset_n is set low, the chip begins to reset. Reset_n may be asynchronous with Sclk or Dclk.

InReady (Output)

InReady is set high when the chip is ready to receive coefficients or input samples; otherwise it is set low. InReady is updated on the rising edge of Sclk.

OutReady (Output)

OutReady is set high when the chip is transmitting output samples; otherwise it is set low. OutReady is aligned with the rising edge of Frame.

InputL (Input)

InputL carries the left channel coefficients and audio samples in serial form. Bit 0 is the sign bit and is transmitted first. Bit 15 is the LSB and is transmitted last. InputL is read on the **falling edge** of Dclk.

InputR (Input)

InputR carries the right channel coefficients and audio samples in serial form. Bit 0 is the sign bit and is transmitted first. Bit 15 is the LSB and is transmitted last. InputR is read on the **falling edge** of Dclk.

OutputL (Output)

OutputL carries the left channel serial output samples. Bit 0 is the sign bit and is transmitted first. Bit 39 is the LSB and is transmitted last. OutputL is updated on the rising edge of Sclk. The output frame starts with the rising edge of Frame and lasts for 40 Sclk cycles.

OutputR (Output)

OutputR carries the right channel serial output samples. Bit 0 is the sign bit and is transmitted first. Bit 39 is the LSB and is transmitted last. OutputR is

updated on the rising edge of Sclk. The output frame starts with the rising edge of Frame and lasts for 40 Sclk cycles.

Frame (Input)

Frame aligns the serial coefficients, input and output samples. Frame is set high for one Dclk cycle when the first bit of the input samples or coefficients is received, and then it is set low.

Note:

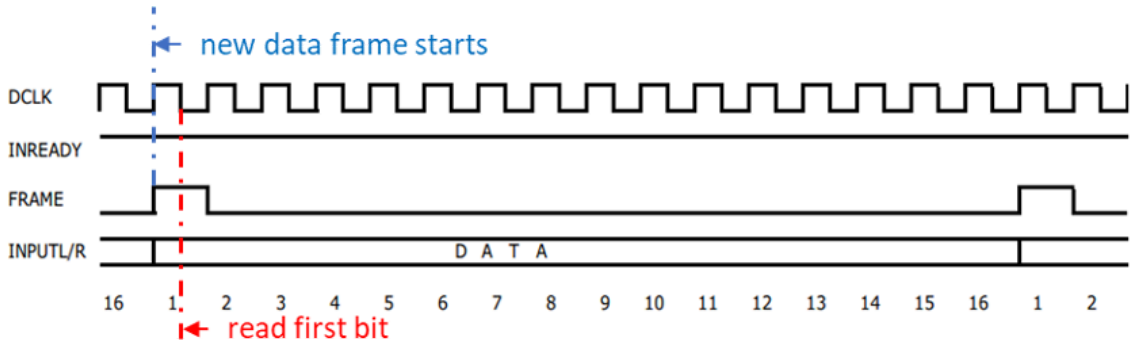
1. Since Coefficients and Rj share the same port with input data, the unused MSB in Rj and coefficients have to be zero padded (to 16 bit as data) by the controller.

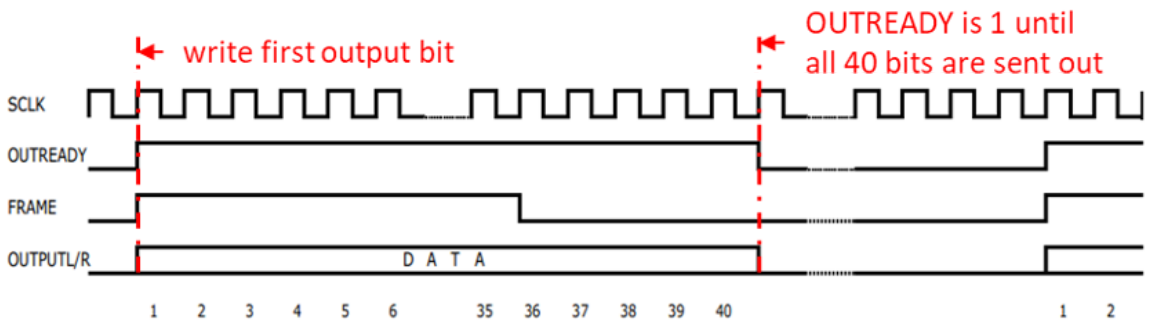
2. InReady is set high by the MSDAP to show that it is ready to receive data. The Frame signal denotes the beginning of an input. If the external controller detects InReady = 0 sent by the chip, the external controller will not transmit Dclk, Frame or any input samples to the chip.

3. When the chip is in sleeping mode, it shall shut down the unnecessary clocks and operations to save energy.

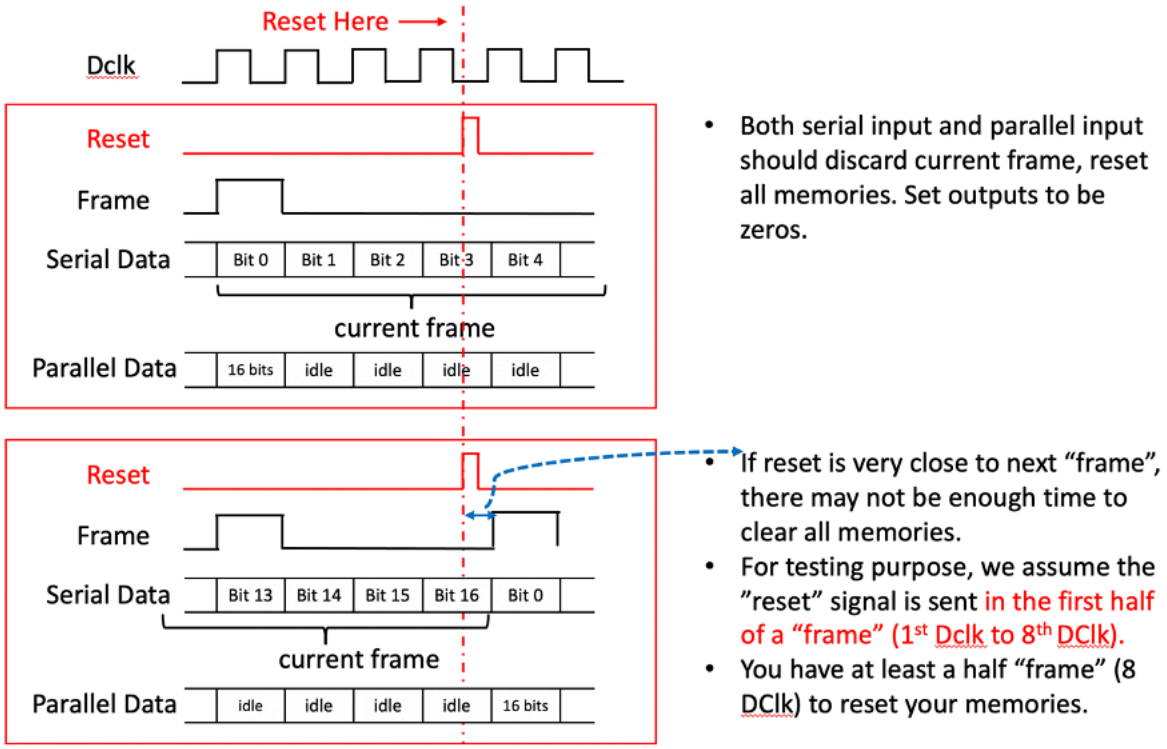
4. External controller should act as soon as possible if it detects InReady = 1 sent by the chip. The continuing inactive time before a new frame will not be more than 16 data clock cycles.

5. You may refer to this timing diagram to read data/write results.





Asynchronized Reset Signal



"Reset" behavior of your module depends on when it is received, as shown in above figure. For the testing purpose, "reset" is assumed to be sent in the first half of "frame". In this way, you should discard the current frame, and set outputs to be zero. (You should include this assumption in your spec)

Input Data Signals

As a real-time audio processing AISC, the length of data inputs to your design is infinite. Your design is restricted to use the following memories which hold a finite amount of data.

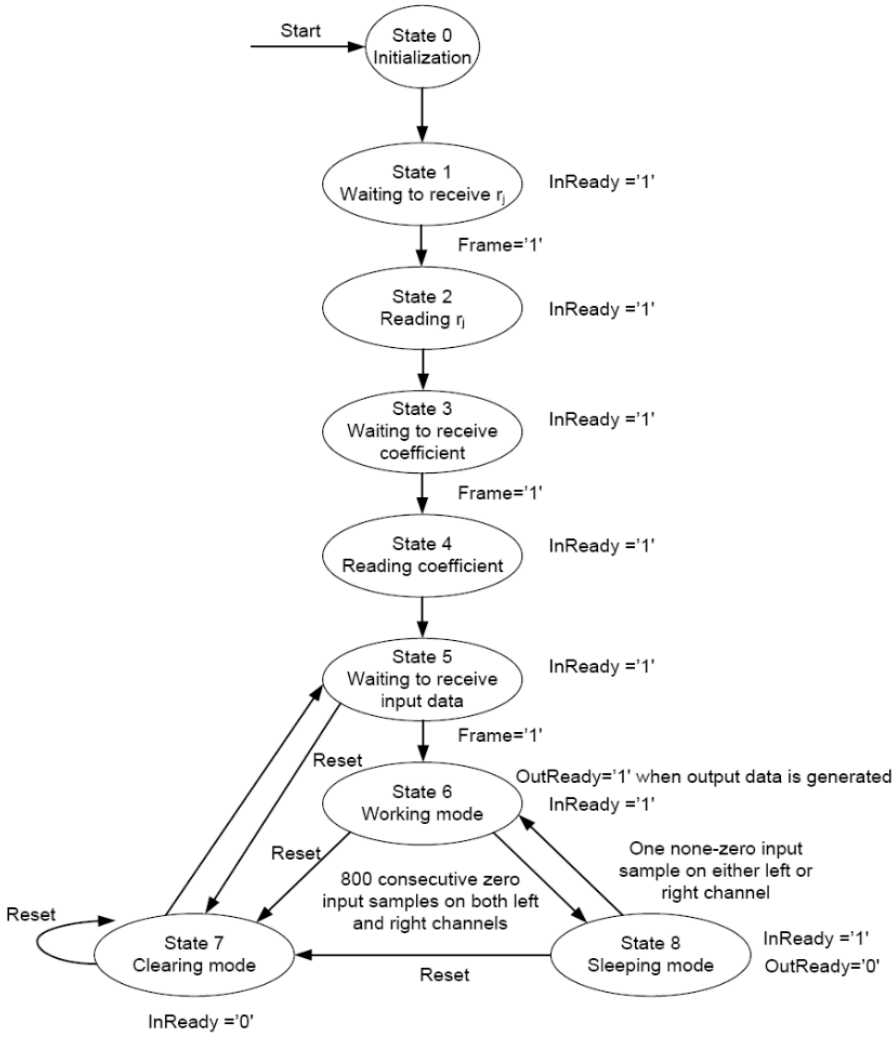
1. The Rj memory: 16bits x 16 (number of Rj) x 2 (channels)
2. The coefficients memory: 16bits x 512 (number of coefficient) x 2 (channels)
3. The data memory: 16bits x 256 (filter order) x 2 (channels)
4. You can use necessary registers to store the intermediate results (like the “u”). However, the number of such registers should be at most 40 bit x 16 (number of u) x 2 (channels).

You might allocate a small amount more registers if you need internal signals to complete the operations. However, allocating a large block of memory to hold all data at once is prohibited.

Clarification of the Sleep Mode

800 consecutive 16bits-zeros for both channels will force the chip entering sleeping mode. Non-zero value detected from either channel will wake up the chip.

Example of FSM State Transition Diagram and State explanations



The above finite state machine works as follows:

State 0 (Initialization): When Start is high, the chip begins the initialization process, including clearing all the memories and registers. When the initialization process is completed, the chip enters State 1.

State 1 (Waiting to receive R_j): In this state, InReady is set high. If Frame is detected to be high, the MSDAP enters State 2.

State 2 (Reading R_j): In this state, the chip reads the Rj values and InReady remains high. Once all Rj values have been loaded, the MSDAP enters State 3.

State 3 (Waiting to receive coefficients): In this state, InReady is set high. If Frame is detected to be high, the MSDAP enters State 4.

State 4 (Reading coefficients): In this state, the chip reads the coefficients. InReady remains high. Once all the coefficients have been loaded, the MSDAP enters State 5.

State 5 (Waiting to receive input data): In this state, InReady is set high. If Frame is detected to be high, the chip enters State 6, or if Reset_n is detected to be low, the MSDAP enters State 7.

State 6 (Working): In this state, the chip continually reads input samples, does the convolution computation, and sends out the computed output data; InReady remains high. If Reset_n is detected low, the chip enters State 7 or if the chip detects 800 consecutive input samples as zero, it enters State 8.

State 7 (Clearing): In this state, InReady is set low. All input and output samples in memories or registers except for Rj and coefficients are cleared to zero. If Reset_n is detected to be low again during this process, the chip will come back to the beginning of this process. The chip will go back to State 5 when completing the task of reset.

State 8 (Sleep): In this state, the chip goes into sleep mode while setting InReady high. If any non-zero input sample on either left or right channel is detected, the chip enters State 6. If Reset_n is detected low, the chip enters State 7.